

Fiche Projet : Le "Smart Fridge & Nutrition Coach" (Projet Fil Rouge)

Format Pédagogique : Apprentissage par projets (Project-Based Learning) en autonomie guidée (~30h).

Stack Technique : FastAPI, Pydantic (validation avancée), Tailwind CSS, Jinja2, httpx, asyncio, SQLite/PostgreSQL, Authentification JWT & Sécurité FastAPI.

APIs Externes : TheMealDB (Recettes, structures éclatées & ingrédients), USDA FoodData Central (Valeurs nutritionnelles brutes & filtres stricts).

PARTIE 1 : CONTEXTE & DETAILS TECHNIQUES (Pour l'intervenant / Mentor)

1. Objectif du Projet

Concevoir et développer de zéro une application web fullstack complète de coaching nutritionnel intelligent. L'utilisateur crée son compte, y renseigne son profil corporel et ses objectifs, saisit les ingrédients disponibles dans son "frigo", et l'application interroge en parallèle **TheMealDB** et **l'USDA** pour calculer les calories réelles ainsi que la répartition exacte des **macronutriments (Protéines ID 1003, Glucides ID 1005, Lipides ID 1004, Énergie ID 1008)** grâce aux identifiants officiels. Le moteur métabolique valide l'équilibre nutritionnel global et génère un plan de repas sur-mesure sécurisé par token JWT.

2. Architecture & Fonctionnalités Clés (Les 4 Piliers du Backend)

A. Le Moteur Métabolique & Validation Pydantic (Séquence 1 - ~6h)

- **Modélisation rigoureuse** : Création de schémas Pydantic stricts pour l'entrée des profils utilisateurs (poids, taille, âge, sexe, niveau d'activité physique, objectif : perte, maintien, prise).
- **Formule de Mifflin-St Jeor** : Implémentation mathématique pure du Métabolisme de Base (BMR) et de la Dépense Énergétique Totale (TDEE).
- **Gestion des deltas caloriques et des macros** : Application dynamique des règles de nutrition (-500 kcal / +300 kcal) et suivi quantitatif des apports en protéines, lipides et glucides.

B. La Pipeline Asynchrone Multi-API & asyncio.gather (Séquence 2 - ~12h)

- **Consommation TheMealDB** : Requêtage par ingrédients du frigo pour récupérer les recettes éligibles.
- **Le défi structurel TheMealDB** : Les ingrédients ne sont pas dans un tableau propre mais

éclatés sur 20 paires de clés indépendantes (strIngredient1, strMeasure1... strIngredient20). Les étudiants doivent écrire un **validateur personnalisé Pydantic** pour aplatir et nettoyer cette structure chaotique.

- **Consommation USDA FoodData Central** : Requêtes POST asynchrones pour chaque ingrédient de la recette afin d'obtenir avec précision les calories et les macronutriments bruts par gramme en ciblant les IDs officiels (Énergie ID 1008, Protéines ID 1003, Glucides ID 1005, Lipides ID 1004).
- **Filtrage avancé anti-plats cuisinés** : Utilisation impérative du paramètre dataType: ["SR Legacy", "Foundation"] pour cibler exclusivement les aliments bruts de référence.

C. Authentification, Sécurité & Gestion des Données (Séquence 3 - ~6h)

- **Inscription & Connexion (JWT)** : Implémentation d'un système de tokens sécurisés (python-jose, hachage des mots de passe) pour authentifier chaque utilisateur.
- **Protection des routes (Depends)** : Utilisation du système d'injection de dépendances de FastAPI pour restreindre l'accès au frigo personnel et aux calculs de nutrition aux seuls utilisateurs connectés.
- **Sécurité web de base** : Configuration propre du CORS et validation stricte des payloads Pydantic.

D. Résilience, Fallbacks et UI Tailwind/Jinja2 (Séquence 4 - ~6h)

- **Le piège de la nomenclature** : Gestion des écarts lexicaux entre l'anglais britannique de TheMealDB ("*Aubergine*") et la taxonomie américaine de l'USDA ("*Eggplant*"). Mise en place d'un dictionnaire de mapping ou de stratégies de secours (*fallback*).
- **Gestion des pannes réseaux** : Gestion des timeouts, des codes 429 (Too Many Requests) et des ingrédients introuvables sans provoquer de crash du serveur FastAPI.
- **Interface Web Ergonomique** : Rendu HTML dynamique via Jinja2 couplé à Tailwind CSS, affichant le profil de l'utilisateur, des jauges caloriques et macro-nutritionnelles dynamiques, ainsi que les suggestions de recettes.

3. ⚠ Les Subtilités & Pièges Pédagogiques

1. **Le piège de la structure TheMealDB (aplatissement des données)** : Transformer dynamiquement les 20 paires de clés éparpillées en une liste propre list[IngredientQuantity] via un validateur Pydantic.
2. **Le piège des plats préparés dans l'USDA** : Forcer l'utilisation de requêtes POST avec filtrage dataType sur les bases officielles brutes (SR Legacy, Foundation) pour éviter les plats industriels.
3. **L'extraction fine des macronutriments** : Exploiter proprement le tableau foodNutrients de l'USDA pour extraire à la fois l'énergie (1008) et les trois grands macros (1003, 1004, 1005).
4. **Le décalage sémantique (Mapping)** : Gérer les différences de termes entre les deux API (ex: Anglais britannique vs Américain).

4. Livrables Attendus pour les Étudiants

- Un dépôt Git propre avec une architecture modulaire.
- Une documentation interactive **Swagger UI** entièrement fonctionnelle et testée.
- Une interface web fluide (Tailjinja) permettant de simuler l'ensemble du parcours utilisateur authentifié avec suivi des macros.
- Un code résilient, documenté et gérant proprement les exceptions asynchrones ainsi que la sécurité des tokens JWT.

Projet Fil Rouge : Smart Fridge & Nutrition Coach

Bienvenue dans ce projet de développement web fullstack ! Pendant ces 30 heures, vous allez concevoir en équipe de 2 une application web intelligente capable de vous concocter des plans de repas sur-mesure en fonction de votre frigo et de vos objectifs corporels.

Votre Mission

Développer une application web moderne de coaching nutritionnel en vous appuyant sur FastAPI, Tailwind CSS et des API externes.

Les APIs Officielles à Intégrer

1. **TheMealDB API (Gratuite, sans clé requise) :**
 - *Rôle* : Rechercher et récupérer des recettes de cuisine à partir des ingrédients disponibles dans votre frigo virtuel.
 - *Le défi* : La structure de réponse de cette API est un peu "bizarre" et demande un bon travail de nettoyage de données.
2. **USDA FoodData Central API (Clé fournie) :**
 - *Rôle* : Interroger la base de données nutritionnelle officielle du gouvernement américain pour chaque ingrédient.
 - *Le défi* : Récupérer non seulement les calories, mais aussi la répartition exacte des **macronutriments** (Protéines, Glucides, Lipides) et cibler uniquement les produits bruts (pas de plats préparés industriels).

Fonctionnalités Attendues dans votre App

1. **Espace Utilisateur & Sécurité** : Inscription, connexion sécurisée via des tokens **JWT**, et protection des pages privées.
2. **Profil & Moteur Métabolique** : Saisie des informations corporelles (poids, taille, âge, sexe, objectif) et calcul automatique des besoins énergétiques et en macronutriments.

3. **Le Frigo Intelligent** : Saisie des ingrédients disponibles chez soi.
4. **Le Moteur de Suggestions** : L'application croise votre frigo avec TheMealDB et l'USDA pour calculer les apports réels de chaque recette proposée.
5. **Interface Graphique** : Un rendu propre, ergonomique et responsive grâce à **Tailwind CSS** et **Jinja2**, avec des jauges visuelles pour les calories et les macros.

Ce que vous devez rendre à la fin

- Un dépôt Git propre avec un historique de commits régulier pour l'équipe.
- Une application fonctionnelle et testable via l'interface web ou la documentation interactive de FastAPI (/docs).
- Une courte démo de votre projet pour présenter vos choix techniques et la manière dont vous avez surmonté les difficultés !

Bon courage à tous, codez bien !